

SCLI — Sistema de Control de Laboratorios e Infraestructura

Defensa del segundo corte · Entrega 3 · Aplicaciones Distribuidas

Isaías Urbina

Infraestructura y clúster CockroachDB E3

Freddy Farinango

Persistencia distribuida (reservas-solicitudes)

Harold Vinueza

Dominio académico y usuarios

Ivan Villamarin

Autenticación y frontend

Equipo PFC: Equipo SCLI

Repositorio: https://github.com/iavillamarin98-pred/Proyecto_Fin_de_Curso_scli (rama feature/entrega-3)

Hoja de ruta y objetivo de la defensa

Objetivo: mostrar el diseño distribuido vigente de SCLI y su evolución hacia consenso Raft en el clúster CockroachDB E3.

1

Arquitectura y alcance

Componentes vigentes y clúster E3

2

Decisiones técnicas

ADR-003 y ADR-004

3

Evidencia y resultados

Código, pruebas y CI

4

Demostración

Ciclo de vida de una solicitud

5

Riesgos y conclusiones

Deuda técnica y plan E4



Problema y alcance del sistema

- Dominio: reservas de laboratorios universitarios (UTEQ).
- Usuarios: estudiantes, docentes y administradores de laboratorio.
- RNF crítico: consistencia serializable bajo concurrencia.
- RNF crítico: tolerancia a la caída de un nodo de datos.

Dominios funcionales

Autenticación

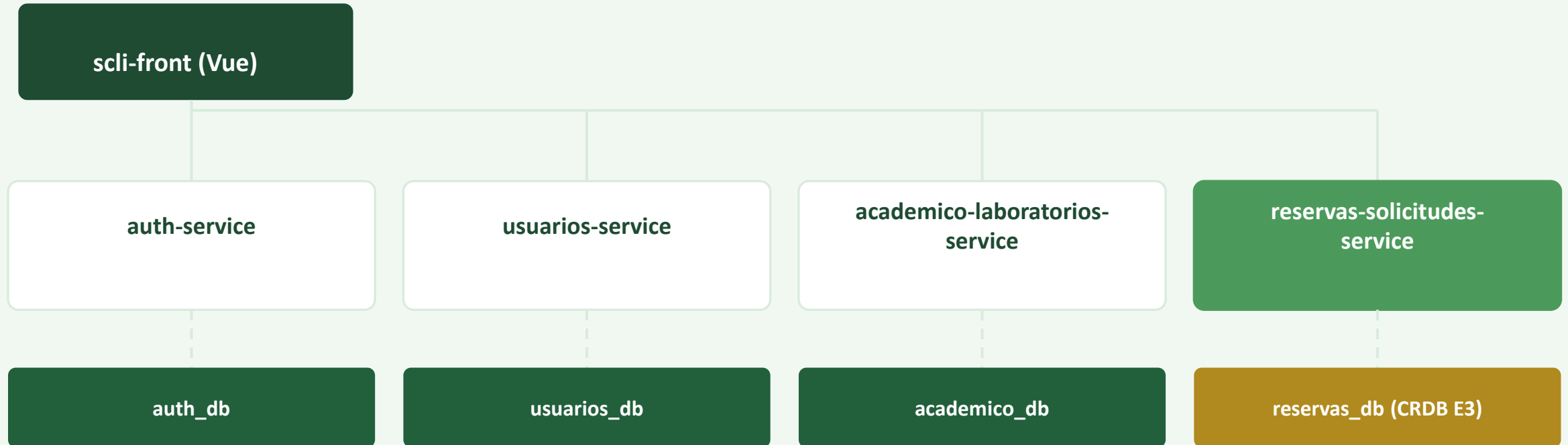
Usuarios

Académico y laboratorios

Reservas y solicitudes

[1] README.md, Estructura general.

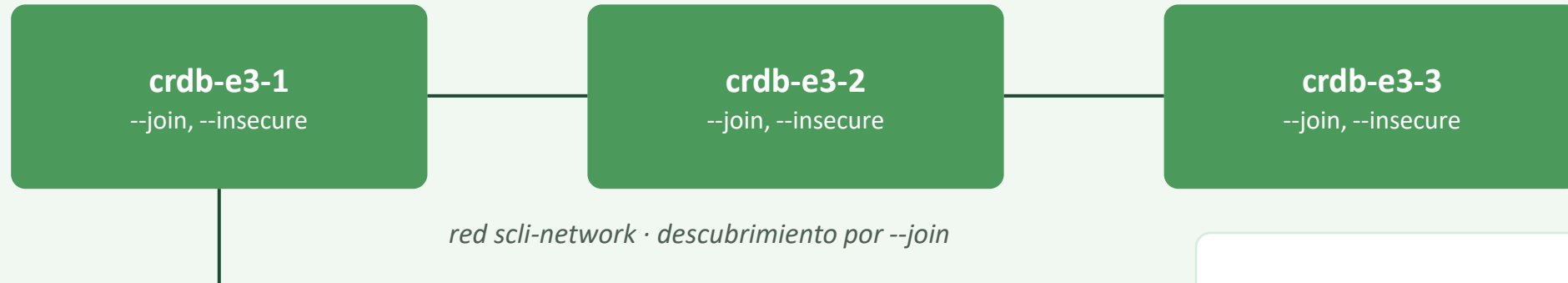
Arquitectura vigente: vista de componentes



- Base de datos independiente por microservicio (sin acoplamiento).
- reservas-solicitudes-service es el único afectado por E3.
- Referencias entre dominios: UUID externos, sin llaves foráneas.

[1] README.md, Estructura general y clúster CockroachDB E3.

Arquitectura vigente: clúster CockroachDB E3



Política aplicada y verificada (ADR-004): ALTER RANGE default CONFIGURE ZONE USING num_replicas = 3;

crdb-e3-init

- cockroach init sobre crdb-e3-1.
- Carga migraciones Flyway (reservas_db).
- Verificado: 6 tablas creadas (diapositiva 13).

[1] docker-compose.yml, servicios crdb-e3-1..3 e init. [4] ADR-004.

Alcance comprometido en E3 y estado real

Planificado

- Clúster de 3 nodos CockroachDB.
- Fragmentación temporal por rango.
- Replicación x3 con consenso Raft.
- Esquema ejecutable (schema.sql).

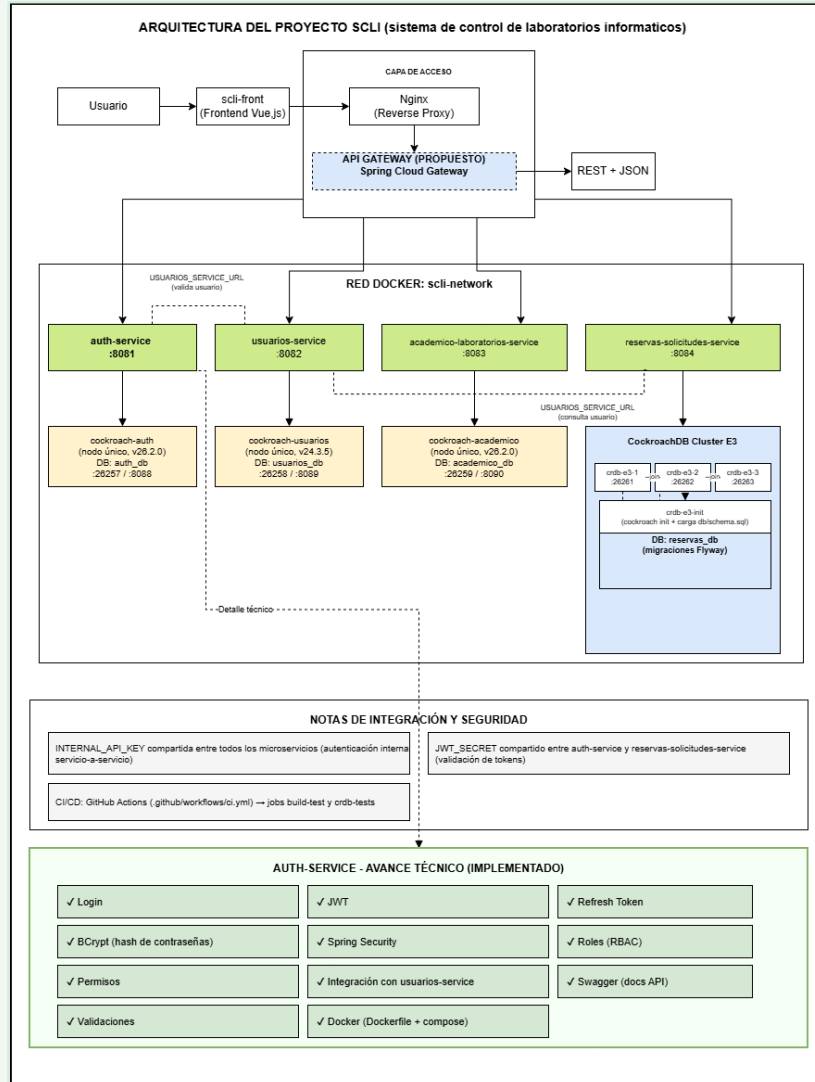
Logrado

- Clúster de 3 nodos desplegado y unido.
- crdb-e3-init inicializa el clúster.
- ADR-003 y ADR-004 redactados y aprobados.
- Esquema migrado (Flyway): 6 tablas creadas en reservas_db.
- num_replicas = 3 aplicado y verificado en el clúster.

No logrado

- Partición temporal (ADR-003) aún sin aplicar sobre las tablas.
- Cobertura de pruebas en reservas-solicitudes-service sigue baja (2/75).
- crdb-tests deshabilitado en CI.

Arquitectura vigente: diagrama completo del sistema



Elementos clave del diagrama

- Capa de acceso: scli-front (Vue) → Nginx (reverse proxy) → API Gateway propuesto (Spring Cloud Gateway) → REST + JSON.
- Red Docker scli-network: auth-service (:8081), usuarios-service (:8082), academico-laboratorios-service (:8083) y reservas-solicitudes-service (:8084), cada uno con su propia base CockroachDB de nodo único.
- CockroachDB Cluster E3: crdb-e3-1/2/3 unidos por --join, inicializados por crdb-e3-init, con la base reservas_db bajo migraciones Flyway.
- Integración y seguridad: INTERNAL_API_KEY compartida entre microservicios, JWT_SECRET compartida entre auth-service y reservas-solicitudes-service, y CI/CD vía GitHub Actions.
- auth-service — avance implementado: login, JWT, refresh token, BCrypt, Spring Security, roles (RBAC), permisos, integración con usuarios-service, Swagger y despliegue con Docker.

Decisión técnica: fragmentación temporal (ADR-003)

Decisión

- PARTITION BY RANGE trimestral en índices temporales.
- Claves UUID existentes se conservan sin cambios.
- Alineación lógica de solicitudes, reservas y bloqueos.

Alternativas descartadas

- Hash del identificador: rompe la poda por fecha.
- Por laboratorio: carga desigual entre laboratorios.
- Por estado: mueve filas entre particiones al cambiar.

Clave de partición por tabla

`solicitudes_reserva` → `fecha_reserva`

`reservas` → `fecha_reserva`

`historial_solicitudes` → `fecha_hora`

`bloqueos_agenda` → `fecha`

Rangos trimestrales, semiabiertos, con particiones de cobertura pasada y futura.

Decisión técnica: consenso Raft y réplicas (ADR-004)

- Política: num_replicas = 3, aplicada y verificada en el clúster E3.
- Cada rango es un grupo Raft; el líder ordena los cambios.
- Una escritura requiere el acuerdo de la mayoría (quórum).

Alternativas descartadas

- 1 réplica: punto único de fallo, sin tolerancia.
- 2 réplicas: quórum de 2; un fallo bloquea la escritura.
- 5+ réplicas: exige más nodos que los 3 disponibles.

Grupo Raft del rango (3 réplicas)



✓ Quórum de 2 de 3 confirma la escritura

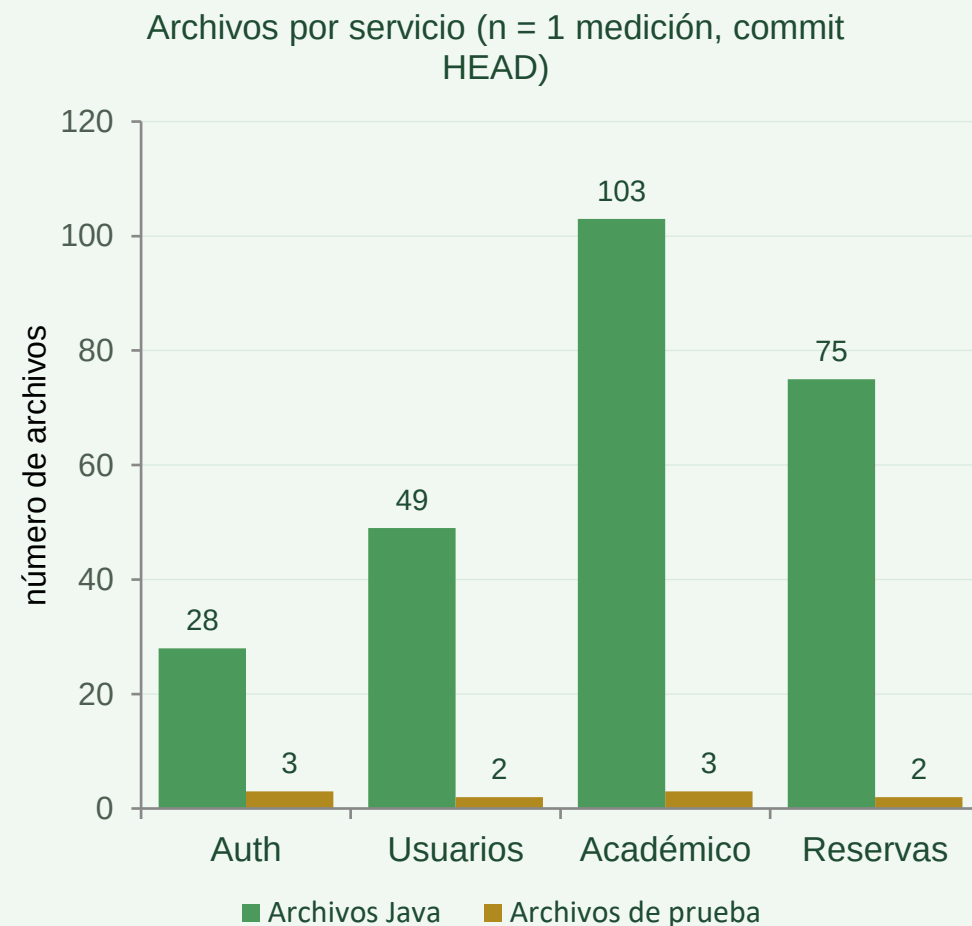
✓ Tolera la caída de 1 nodo sin perder quórum (verificado, diap. 14)

! 2 nodos caídos = sin escritura posible

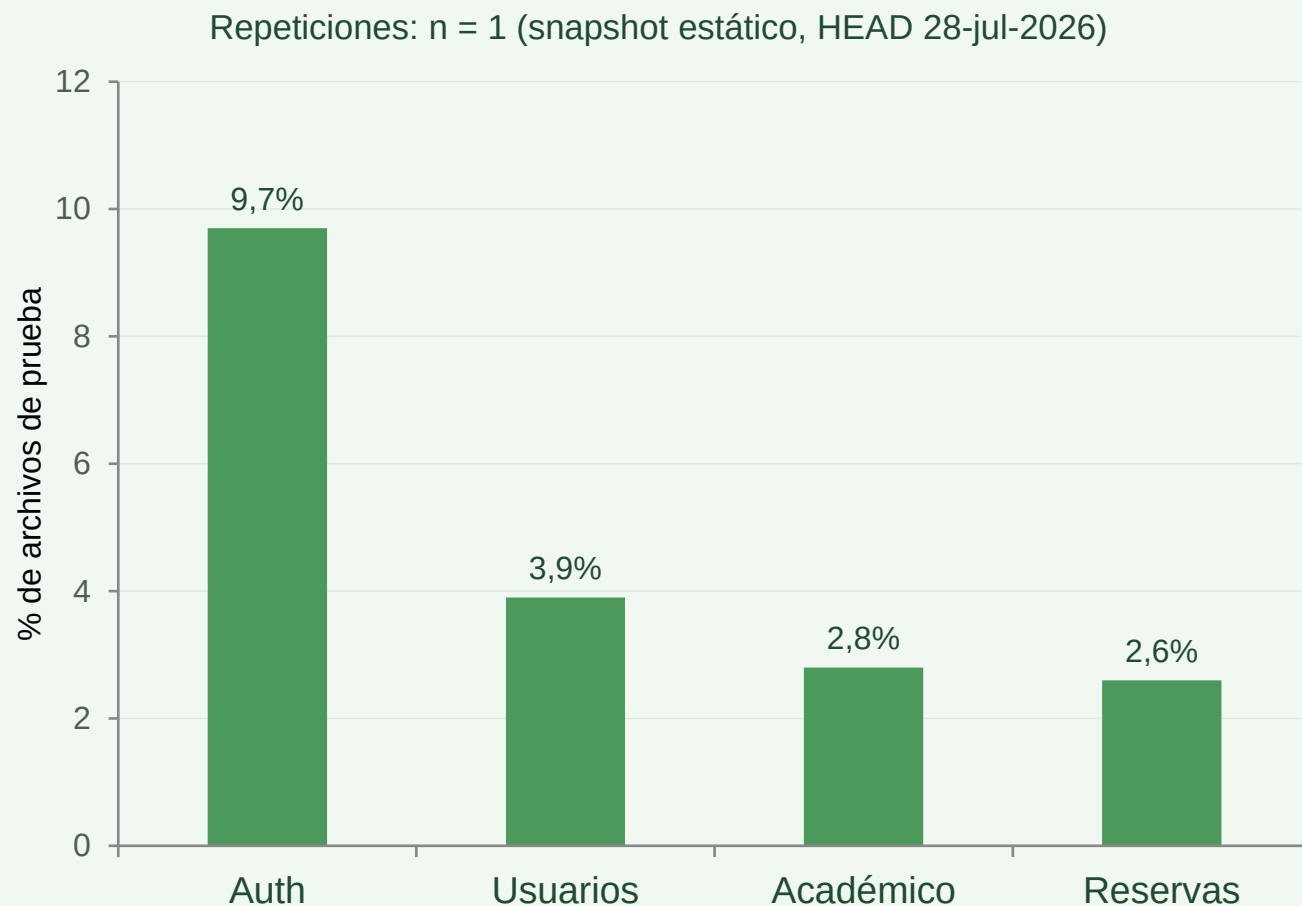
Evidencia de implementación

- 4 microservicios Spring Boot + frontend Vue + módulo Spark.
- 265 archivos Java en los 4 microservicios (rama actual).
- reservas-solicitudes-service expone 18 endpoints REST.
- CI: matriz build-test por servicio con CockroachDB efímero.

```
GET /api/v1/solicitudes/{id}/historial
POST /api/v1/solicitudes/{id}/aprobar
```



Resultados: proporción de pruebas por servicio



Lectura del resultado

- auth-service tiene la mayor proporción de pruebas.
- reservas-solicitudes-service, el mayor volumen, es el
- menos cubierto en proporción.
- CI ejecuta build-test por servicio en cada push.
- crdb-tests permanece deshabilitado (Paso 2 pendiente).

Medición propia sobre `.github/workflows/ci.yml` y conteo de archivos, HEAD 28-jul-2026.

Resultados: tolerancia a fallos del diseño de réplicas

Réplicas (n)	Quórum (mayoría)	Fallos tolerados	Veredicto
1	1	0	Descartado
2	2	0	Descartado
3	2	1	Adoptado (ADR-004)
5	3	2	No aplica: 3 nodos

Unidades: número de réplicas por rango. Repeticiones: 4 configuraciones evaluadas analíticamente.

- Con 3 nodos, 3 réplicas es el mínimo que tolera 1 fallo.
- El quórum de 2 se mantiene con un nodo caído.

Resultados: verificación empírica del clúster E3

```

PS C:\Users\HP\IdeaProjects\Proyecto_Fin_de_Curso_scli> docker exec -it scli-crdb-e3-1 cockroach sql --insecure --database=cockroach --target=
TION FOR RANGE default;"
target | raw_config_sql
-----|-----
RANGE default | ALTER RANGE default CONFIGURE ZONE USING
              | range_min_bytes = 134217728,
              | range_max_bytes = 536870912,
              | gc.ttlseconds = 14400,
              | num_replicas = 3,
              | constraints = '{}',
              | lease_preferences = '{}'
(1 row)
Time: 4ms

```

SHOW ZONE CONFIGURATION: num_replicas = 3 verificado.

```

PS C:\Users\HP\IdeaProjects\Proyecto_Fin_de_Curso_scli> docker exec -it scli-crdb-e3-1 cockroach sql --insecure --database=reservas
schema_name | table_name | type | owner | estimated_row_count | locality
-----|-----|-----|-----|-----|-----
public | bloques_agenda | table | root | 0 | NULL
public | configuraciones_reserva | table | root | 0 | NULL
public | flyway_schema_history | table | root | 1 | NULL
public | historial_solicitudes | table | root | 0 | NULL
public | reservas | table | root | 0 | NULL
public | solicitudes_reserva | table | root | 0 | NULL
(6 rows)
Time: 75ms

```

SHOW TABLES: 6 tablas creadas, incl. flyway_schema_history.

Qué confirma la evidencia

- Política num_replicas = 3 (ADR-004) verificada con SHOW ZONE CONFIGURATION sobre el clúster real.
- Migraciones Flyway ejecutadas: 6 tablas creadas en reservas_db, incluida flyway_schema_history.
- seeds.sql creado y aplicado: datos iniciales de configuraciones_reserva cargados en el clúster.
- cockroach node status confirma los 3 nodos (crdb-e3-1/2/3) activos (is_live = true) tras el despliegue.
- Capturas propias, terminal VS Code / PowerShell, sesión del 28-jul-2026 sobre el clúster E3 (docker exec).

Capturas propias: docker exec cockroach sql / node status, clúster E3, 28-jul-2026.

Resultados: latencia y tolerancia a fallos con datos reales

```
PS C:\Users\HP\IdeaProjects\Proyecto_Fin_de_Curso_scli> docker exec -it scli-crdb-e3-1 cockroach node status --insecure
id | address | sql_address | build | started_at | updated_at | locality | attrs | is_ava
able | is_live
-----
1 | crdb-e3-1:26257 | crdb-e3-1:26257 | v26.2.0 | 2026-07-28 19:48:31.225486 +0000 UTC | 2026-07-28 22:41:57.675019 +0000 UTC | [] | true
2 | crdb-e3-3:26257 | crdb-e3-3:26257 | v26.2.0 | 2026-07-28 19:48:31.128758 +0000 UTC | 2026-07-28 22:41:57.582211 +0000 UTC | [] | true
3 | crdb-e3-2:26257 | crdb-e3-2:26257 | v26.2.0 | 2026-07-28 19:48:31.051986 +0000 UTC | 2026-07-28 22:41:44.295766 +0000 UTC | [] | false
(3 rows)

What's next:
```

docker stop crdb-e3-2: node status confirma 2/3 nodos activos.

```
PS C:\Users\HP\IdeaProjects\Proyecto_Fin_de_Curso_scli> docker exec -it scli-crdb-e3-1 cockroach node status --insecure
id | address | sql_address | build | started_at | updated_at | locality | attrs | is_ava
able | is_live
-----
1 | crdb-e3-1:26257 | crdb-e3-1:26257 | v26.2.0 | 2026-07-28 19:48:31.225486 +0000 UTC | 2026-07-28 22:49:52.998618 +0000 UTC | [] | true
2 | crdb-e3-3:26257 | crdb-e3-3:26257 | v26.2.0 | 2026-07-28 22:49:09.244672 +0000 UTC | 2026-07-28 22:49:49.018474 +0000 UTC | [] | true
3 | crdb-e3-2:26257 | crdb-e3-2:26257 | v26.2.0 | 2026-07-28 22:49:08.432478 +0000 UTC | 2026-07-28 22:49:48.496492 +0000 UTC | [] | true
(3 rows)

What's next:
```

Tras reiniciar los nodos: 3/3 activos (recuperación completa).

Medición de latencia bajo caída de nodo (n = 1 por etapa)

- Carga de datos: lote 1 de 10 000 registros (22 ms, motor SQL) escalado a 100 000 registros (180 ms, motor SQL) en solicitudes_reserva.
- Latencia extremo a extremo antes de la caída (Measure-Command, incluye arranque de docker exec): 2 604 ms.
- Con 1 nodo caído (2/3 activos): 2 681 ms — sin caída del servicio, quórum mantenido.
- Tras la recuperación (3/3 activos): 2 906 ms — variación atribuible al rebalanceo de rangos, no a pérdida de datos.
- Con 2 nodos caídos (1/3 activos) se pierde el quórum, confirmando en clúster real el límite analítico de la diapositiva 12.

Unidades: ms. n = 1 por etapa. Fuente: cockroach sql y PowerShell Measure-Command, 28-jul-2026.

Ancla de la demostración: objetivo y pasos

Objetivo: iniciar sesión, emitir un JWT válido y verificar rol y permisos en `/api/v1/auth/me`.

1

Login

Ingresa admin en scli-front
(localhost:5173).

2

Emitir JWT

POST `/api/v1/auth/login` devuelve
accessToken (Bearer).

3

Sesión

El frontend guarda el token y
redirige a `/dashboard`.

4

Verificar

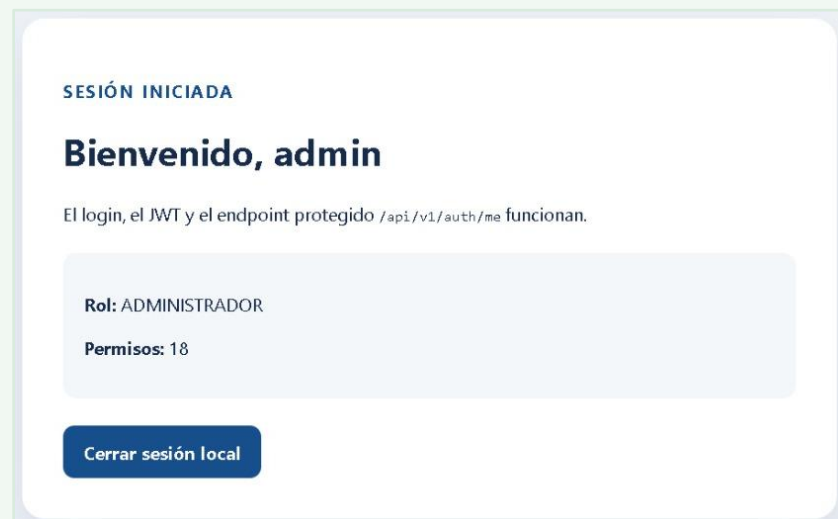
GET `/api/v1/auth/me` confirma rol
y permisos.

Resultado esperado: sesión iniciada, JWT Bearer válido, rol ADMINISTRADOR y sus permisos.

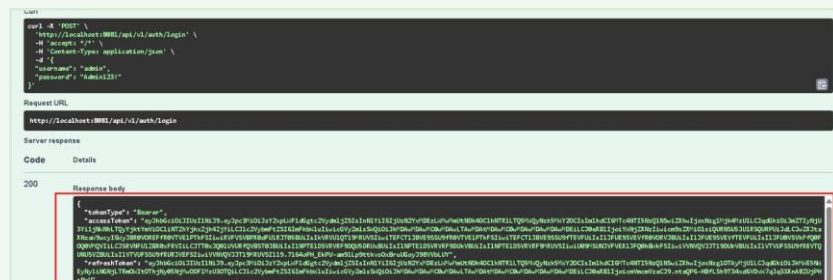
Respaldo: capturas reales de la demo grabada, incrustadas en la diapositiva siguiente.

[6] auth-service: SecurityConfig.java, JwtService.java y AuthController.java.

Lectura del resultado de la demostración



Dashboard real: rol ADMINISTRADOR y 18 permisos confirmados.



curl + respuesta 200: tokenType Bearer, accessToken y refreshToken.

Qué confirma la evidencia

- Bienvenido, admin: sesión persistida y validada en el frontend (scli-front).
- Rol ADMINISTRADOR y 18 permisos devueltos por /api/v1/auth/me, acorde al modelo RBAC diseñado.
- El login vía Swagger devuelve tokenType Bearer, accessToken y refreshToken firmados (JWT).
- auth-service, usuarios-service y scli-front interoperan correctamente de extremo a extremo.
- **Vincula con:** seguridad y control de acceso del sistema (auth-service, diapositiva 7).

Capturas propias: login, dashboard, Swagger y curl reales, sesión grabada 28-jul-2026.

Equipo SCLI · EX-PFC-E3-DIAP · SCLI — Sistema de Control de Laboratorios e Infraestructura



Riesgos, deuda técnica y plan hacia E4

! Partición temporal (ADR-003) aún sin aplicar

Mitigación / plan E4: Aplicar PARTITION BY RANGE sobre el esquema ya migrado, en Paso 2.

! crdb-tests deshabilitado en CI

Mitigación / plan E4: Habilitar la variable CRDB_E3_TESTS_ENABLED al cerrar Paso 2.

! Cobertura de pruebas desigual (2/75 en reservas)

Mitigación / plan E4: Ampliar Testcontainers para transiciones críticas.



Conclusiones propias del equipo

1

Fragmentación y replicación son decisiones complementarias, pero solo la replicación está verificada en el clúster real; la fragmentación queda para E4.

2

Aislar bases de datos por microservicio evitó acoplamiento, pero trasladó toda la complejidad distribuida al clúster E3.

3

La cobertura desigual de pruebas es el mayor riesgo de calidad heredado hacia la entrega final.

Declaración de uso de IA generativa

- **Herramienta:** Claude (Anthropic), asistente de IA generativa de texto.
- **Propósito:** revisar el mazo contra la rúbrica EX-PFC-E3-DIAP, redactar y maquetar diapositivas de evidencia a partir de capturas propias del equipo.
- **Diapositivas asistidas:** 13 y 14 (evidencia empírica y latencia, nuevas), y edición de texto en 5, 6, 9, 15 y 16 para reflejar el estado real verificado.
- **Alcance:** todo el código, las decisiones ADR, las mediciones y las capturas de terminal son trabajo propio del equipo; la IA no generó datos ni resultados.

Referencias

- [1] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm (Raft),” in Proc. USENIX Annu. Tech. Conf. (ATC), 2014.
- [2] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, Dec. 12, 2024.
- [3] Equipo SCLI, “README.md — Proyecto_Fin_de_Curso_scli,” repositorio del equipo, 2026.
- [4] Equipo SCLI, “ADR-003: Fragmentación temporal de tablas transaccionales,” documento de arquitectura, 2026.
- [5] Equipo SCLI, “ADR-004: Política de réplicas y consenso Raft en clúster CockroachDB E3,” documento de arquitectura, 2026.